

实验六 CPU 综合设计

一、实验目的

- 1 掌握复杂系统设计方法。
- 2 深刻理解计算机系统硬件原理。

二、实验内容

1) 设计一个基于 MIPS 指令集的 CPU，支持以下指令：{add, sub, addi, lw, sw, beq, j, nop};

2) CPU 需要包含寄存器组、RAM 模块、ALU 模块、指令译码模块;

3) 该 CPU 能运行基本的汇编指令; (D~C+)

以下为可选内容:

4) 实现多周期 CPU (B~B+);

5) 实现以下高级功能之一 (A~A+):

(1) 实现 5 级流水线 CPU;

(2) 实现超标量;

(3) 实现 4 路组相联缓存;

可基于 RISC V、ARM 指令集实现。

如发现代码为抄袭代码，成绩一律按不及格处理。

三、实验要求

编写相应测试程序，完成所有指令测试。

四、实验代码及结果

这个实验中，我实现了一个基于 RISC-V 指令集的多周期 CPU，并且实现了两个高级功能：五级流水线和四路组相联缓存。

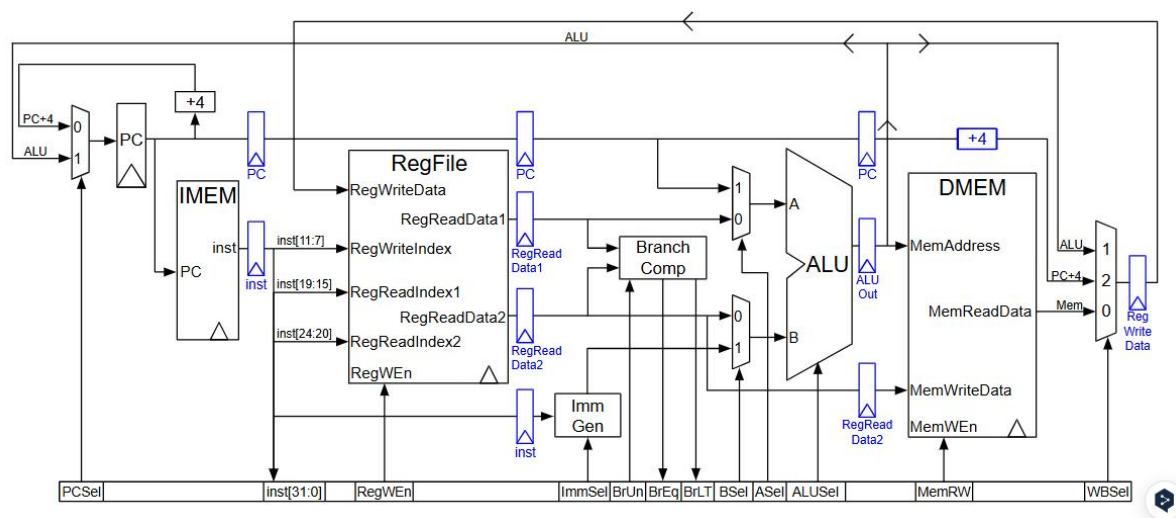
这个 CPU 支持以下指令：

```

add rd, rs1, rs2
mul rd, rs1, rs2
sub rd, rs1, rs2
sll rd, rs1, rs2
mulh rd, rs1, rs2
mulhu rd, rs1, rs2
slt rd, rs1, rs2
xor rd, rs1, rs2
srl rd, rs1, rs2
sra rd, rs1, rs2
or rd, rs1, rs2
and rd, rs1, rs2
lw rd, offset(rs1)
addi rd, rs1, imm
slli rd, rs1, imm
slti rd, rs1, imm
xori rd, rs1, imm
srli rd, rs1, imm
srai rd, rs1, imm
ori rd, rs1, imm
andi rd, rs1, imm
sw rs2, offset(rs1)
beq rs1, rs2, offset
bne rs1, rs2, offset
blt rs1, rs2, offset
bge rs1, rs2, offset
bltu rs1, rs2, offset
bgeu rs1, rs2, offset
auipc rd, offset
lui rd, offset
jal rd, imm
jalr rd, rs1, imm

```

这个 CPU 整体的数据通路大致如下（这里未包含数据冒险、控制冒险处理电路）：
（本图来自于 UC Berkeley CS61C 课程的 ppt）



其中 DMem（数据存储器）里内置了一个四路组相联缓存；最下方的控制逻辑信

号与指令解码阶段进行解码，并各设置了一个流水线寄存器用于保存信号。
以下为代码：

(1) 顶层代码：

```
module CPU_Datapath(  
    input clk,  
    input rst  
);  
  
    reg [31:0] PC_in;  
    wire [31:0] PC_out;  
    wire [31:0] PC_ID, PC_EX, PC_M;  
  
    wire [31:0] instr;  
    reg [31:0] instr_ID, instr_IF;  
    wire [31:0] instr_F;  
    wire [31:0] instr_EX, instr_M, instr_WB;  
  
    wire [31:0] RD1, RD2;  
    reg [31:0] RD1_ID, RD2_ID;  
    wire [31:0] RD1_F, RD2_F;  
    reg [31:0] RD1_EX, RD2_EX;  
    wire [31:0] RD2_M;  
  
    wire [31:0] ALUResult_EX;  
    wire [31:0] ALUResult_M;  
  
    reg [31:0] WD3_M;  
    wire [31:0] WD3_WB;  
  
    wire WE3_ID;  
    wire [2:0] ImmSel_ID;  
    wire BrUn_ID, BrEq, BrLt, BSel_ID, ASel_ID;  
    wire [3:0] ALUSel_ID;  
    wire MemRW_ID, MemC_ID;  
    wire [1:0] WBSel_ID;  
  
    wire PCSel_EX, WE3_EX;  
    wire [2:0] ImmSel_EX;  
    wire BrUn_EX, BSel_EX, ASel_EX;  
    wire [3:0] ALUSel_EX;  
    wire MemRW_EX, MemC_EX;  
    wire [1:0] WBSel_EX;  
  
    wire WE3_M;
```

```

wire MemRW_M, MemC_M;
wire [1:0] WBSel_M;

wire WE3_WB;
wire [1:0] WBSel_WB;

wire [31:0] Imm;

reg [31:0] ALU_A, ALU_B;
wire [31:0] ALUResult;
wire [31:0] MemReadData;
reg [1:0] forward_A, forward_B;
reg forward_A_ID, forward_B_ID;

wire [31:0] X1, X2, X3, X4;
initial begin
    PC_in = 0;
    instr_IF = 32'h0000_0013;
    instr_ID = 32'h0000_0013;
    RD1_EX = 0;
    RD2_EX = 0;
    WD3_M = 0;
    ALU_A = 0;
    ALU_B = 0;
    forward_A = 0;
    forward_B = 0;
    forward_A_ID = 0;
    forward_B_ID = 0;
end

// IF Stage

IMem IMem_1(PC_out, instr);
Register PCReg_IF(PC_out, PC_in, rst, clk);
always @(*) begin
    case (PCSel_EX)
        1'b0: begin
            PC_in = PC_out + 4;
            instr_IF = instr;
        end
        1'b1: begin
            PC_in = ALUResult_EX;
            instr_IF = 32'h0000_0013; //nop
        end
    end
end

```

```

        default: begin
            PC_in = PC_out + 4;
            instr_IF = instr;
        end
    endcase
end

// IF Stage
// ID Stage
ControlLogic_ID ConLogi_1(instr_ID, WE3_ID, ImmSel_ID, BrUn_ID,
BSel_ID, ASel_ID, ALUSel_ID, MemRW_ID, MemC_ID, WBSel_ID);
RegFile RegFile_1(clk, WE3_WB, instr_ID[19:15], instr_ID[24:20],
instr_WB[11:7], WD3_WB, RD1, RD2, X1, X2, X3, X4);
Register PCreg_ID(PC_ID, PC_out, rst, clk);
Register instrReg_ID(instr_F, instr_IF, rst, clk);
always @(*) begin
    case (PCSel_EX)
        1'b0: begin
            instr_ID = instr_F;
        end
        1'b1: begin
            instr_ID = 32'h0000_0013; //nop
        end
        default: begin
            instr_ID = instr_F;
        end
    endcase
end
always @(*) begin
    if (instr_WB[11:7] == instr_ID[19:15] && WE3_WB &&
instr_WB[11:7] != 0)
        forward_A_ID = 1'b1;
    else
        forward_A_ID = 1'b0;

    if (instr_WB[11:7] == instr_ID[24:20] && WE3_WB &&
instr_WB[11:7] != 0)
        forward_B_ID = 1'b1;
    else
        forward_B_ID = 1'b0;

    case (forward_A_ID)
        1'b0: RD1_ID = RD1;           // 从寄存器文件
        1'b1: RD1_ID = WD3_WB;       // 从 WB 阶段
    endcase
end

```

```

        default: RD1_ID = 32'bx;
    endcase

    case (forward_B_ID)
        1'b0: RD2_ID = RD2;           // 从寄存器文件
        1'b1: RD2_ID = WD3_WB;       // 从 WB 阶段
        default: RD2_ID = 32'bx;
    endcase
end

// EX Stage
ImmGen ImmGen_1(instr_EX, ImmSel_EX, Imm);
ALU ALU_1(ALUSel_EX, ALU_A, ALU_B, ALUResult_EX);
ControlLogic_EX ControlLogi_EX(instr_EX, BrEq, BrLt, PCSel_EX);
BranchComp BranchComp_1(RD1_EX, RD2_EX, BrUn_EX, BrEq, BrLt);
Register PCReg_EX(PC_EX, PC_ID, rst, clk);
Register instrReg_EX(instr_EX, instr_ID, rst, clk);
Register WE3Reg_EX(WE3_EX, WE3_ID, rst, clk);
Register ImmSelReg_EX(ImmSel_EX, ImmSel_ID, rst, clk);
Register BrUnReg_EX(BrUn_EX, BrUn_ID, rst, clk);
Register BSelReg_EX(BSel_EX, BSel_ID, rst, clk);
Register ASelReg_EX(ASel_EX, ASel_ID, rst, clk);
Register ALUSelReg_EX(ALUSel_EX, ALUSel_ID, rst, clk);
Register MemRWReg_EX(MemRW_EX, MemRW_ID, rst, clk);
Register MemCReg_EX(MemC_EX, MemC_ID, rst, clk);
Register WBSelReg_EX(WBSel_EX, WBSel_ID, rst, clk);
Register RD1Reg_EX(RD1_F, RD1_ID, rst, clk);
Register RD2Reg_EX(RD2_F, RD2_ID, rst, clk);

always@(*) begin
    if (instr_M[11:7] == instr_EX[19:15] && WE3_M &&
instr_M[11:7] != 0)
        forward_A = 2'b10;
    else if (instr_WB[11:7] == instr_EX[19:15] && WE3_WB &&
instr_WB[11:7] != 0)
        forward_A = 2'b01;
    else
        forward_A = 2'b00;

    if (instr_M[11:7] == instr_EX[24:20] && WE3_M &&
instr_M[11:7] != 0)
        forward_B = 2'b10;
    else if (instr_WB[11:7] == instr_EX[24:20] && WE3_WB &&
instr_WB[11:7] != 0)

```

```

        forward_B = 2'b01;
    else
        forward_B = 2'b00;

    case (forward_A)
        2'b00: RD1_EX = RD1_F;           // 从寄存器文件
        2'b10: RD1_EX = WD3_M;           // 从 MEM 阶段
        2'b01: RD1_EX = WD3_WB;          // 从 WB 阶段
        default: RD1_EX = 32'bx;
    endcase

    // ALU_B 的数据来源
    case (forward_B)
        2'b00: RD2_EX = RD2_F;           // 从寄存器文件
        2'b10: RD2_EX = WD3_M;           // 从 MEM 阶段
        2'b01: RD2_EX = WD3_WB;          // 从 WB 阶段
        default: RD2_EX = 32'bx;
    endcase
end
always @(*) begin
    case (ASel_EX)
        1'b1: begin ALU_A = PC_EX; end
        1'b0: begin ALU_A = RD1_EX; end
    endcase
    case (BSel_EX)
        1'b0: begin ALU_B = RD2_EX; end
        1'b1: begin ALU_B = Imm; end
    endcase
end

// EX Stage
// Mem Stage

DMem DMem_1(ALUResult_M, RD2_M, MemC_M, MemRW_M, clk, MemReadData);

Register PCReg_M(PC_M, PC_EX, rst, clk);
Register instrReg_M(instr_M, instr_EX, rst, clk);
Register WE3Reg_M(WE3_M, WE3_EX, rst, clk);
Register MemRWReg_M(MemRW_M, MemRW_EX, rst, clk);
Register MemCReg_M(MemC_M, MemC_EX, rst, clk);
Register WBSelReg_M(WBSel_M, WBSel_EX, rst, clk);
Register ALUResultReg_M(ALUResult_M, ALUResult_EX, rst, clk);
Register RD2Reg_M(RD2_M, RD2_EX, rst, clk);

```

```

always @(*) begin
    case (WBSel_M)
        2'b00: begin WD3_M = MemReadData; end
        2'b01: begin WD3_M = ALUResult_M; end
        2'b10: begin WD3_M = PC_M + 4; end
        default: WD3_M = 32'bx;
    endcase
end

// Mem Stage
// WB Stage
Register instrReg_WB(instr_WB, instr_M, rst, clk);
Register WE3Reg_WB(WE3_WB, WE3_M, rst, clk);
Register WBSelReg_WB(WBSel_WB, WBSel_M, rst, clk);
Register WD3Reg_WB(WD3_WB, WD3_M, rst, clk);

// WB Stage

endmodule

```

这是顶层的 CPU 代码，分为五个阶段：IF(指令读取)、ID(指令解码)、EX(运行)、M(内存存取)、WB(写回)。

为了解决数据冒险，这里设计了转发逻辑。因为从读取指令到写回寄存器文件之间差的循环次数最多为三，所以我们可以分循环次数处理数据冒险。

①若指令之间差三个循环，即读寄存器指令在 ID 阶段，写寄存器在 WB 阶段，则如果写回阶段指令的目标寄存器与解码阶段的指令的源寄存器一致，则将写回阶段的结果提前转发到解码阶段。

②若指令之间差两个循环，即读寄存器指令在 ID 阶段/EX 阶段，写寄存器指令在 M 阶段/WB 阶段，这里统一设置在 EX/WB 阶段处理，提前将 WB 阶段的结果转发回 EX 阶段。

③若指令之间差一个循环，即 ID/EX 或者 EX/M，同样这里统一在 EX/M 阶段处理，将 M 阶段的结果转发回 EX 阶段。

如此便能够不用插入气泡的解决数据冒险。

为了解决控制冒险，增加了分支预测单元。这里的分支预测逻辑较为简单，统一假设不采用分支，若不采用分支，则不用插入气泡，若采用分支，则将运行到 IF 阶段和 ID 阶段的指令统一替换为气泡。如此以来控制冒险得到了解决。

(2) IMem(指令存储器)

```

`define DATA_WIDTH 32

module IMem(
    input [31:0] A,
    output [`DATA_WIDTH-1:0] RD

```



```

);
parameter IMEM_SIZE = 1024;
reg [`DATA_WIDTH-1:0] RAM[IMEM_SIZE-1:0];
initial
    $readmemh("c:\\XJTU\\test.dat", RAM);
assign RD = RAM[A>>2];
endmodule

```

这里统一读取硬盘中的.dat 文件作为每次运行的指令，将其存储在 RAM 中。

(3) Register 寄存器

```

module Register(
    Q, D, OE, CLK
);
parameter N = 32;
output reg [N-1:0] Q;
input [N-1:0] D;
input OE, CLK;
initial begin
    Q = 32'b0;
end
always @(posedge CLK or posedge OE)
    if(OE) Q <= 32'b0;
    else Q <= D;
endmodule

```

这里写了单个寄存器的代码，用于 PC 寄存器和流水线寄存器等 CPU 需要的寄存器。

(4) Control Logic 控制逻辑信号

```

module ControlLogic_ID(
    input [31:0] instr,
    output RegWEn,
    output [2:0] ImmSel,
    output BrUn, BSel, ASel,
    output [3:0] ALUSel,
    output MemRW, MemC,
    output [1:0] WBSel
);
MainDsc MainDsc_1(instr, RegWEn, ImmSel, BrUn, BSel, ASel, ALUSel, MemRW, MemC,
WBSel);
endmodule
module MainDsc(

```

```

input [31:0] instr,
output RegWEn,
output [2:0] ImmSel,
output BrUn, BSel, ASel,
output [3:0] ALUSel,
output MemRW, MemC,
output [1:0] WBSel
);
reg [6:0] op;
reg [2:0] funct3;
reg [6:0] funct7;
reg [14:0] Control;
assign {RegWEn, ImmSel, BrUn, ASel, BSel, ALUSel, MemRW, WBSel, MemC} = Control;
always@(*) begin
    op = instr[6:0];
    funct3 = instr[14:12];
    funct7 = instr[31:25];
    case (op)
        7'h13 : begin // I Type
            case (funct3)
                3'h0: begin //addi
                    Control = 15'b1000001000000010;
                end
                3'h1: begin //slli
                    Control = 15'b100000100010010;
                end
                3'h2: begin //slti
                    Control = 15'b100000100100010;
                end
                3'h4: begin //xori
                    Control = 15'b100000101000010;
                end
                3'h5: begin
                    case (funct7)
                        7'h00: begin //srli
                            Control = 15'b100000101010010;
                        end
                        7'h20: begin //srai

```

```

        Control = 15'b100000111010010;
    end
    default: begin
        Control = 15'bz;
    end
endcase
end
3'h6: begin //ori
    Control = 15'b100000101100010;
end
3'h7: begin //andi
    Control = 15'b100000101110010;
end
default: begin
    Control = 15'bz;
end
endcase
end
7'h33 : begin // R Type
    case (funct3)
        3'h0: begin
            case (funct7)
                7'h00: begin //add
                    Control = 15'b1000000000000010;
                end
                7'h01: begin //mul
                    Control = 15'b100000010000010;
                end
                7'h20: begin //sub
                    Control = 15'b100000011000010;
                end
                default: begin
                    Control = 15'bz;
                end
            endcase
        end
    endcase
end
3'h1: begin
    case (funct7)

```

```

        7'h00: begin //sll
            Control = 15'b100000000010010;
        end
        7'h01: begin //mulh
            Control = 15'b100000010010010;
        end
        default: begin
            Control = 14'bz;
        end
    endcase
end
3'h2: begin //slt
    Control = 15'b100000000100010;
end
3'h3: begin //mulhu
    Control = 15'b100000010110010;
end
3'h4: begin //xor
    Control = 15'b100000001000010;
end
3'h5: begin
    case (funct7)
        7'h00: begin //srl
            Control = 15'b100000001010010;
        end
        7'h20: begin //sra
            Control = 15'b100000011010010;
        end
        default: begin
            Control = 15'bz;
        end
    endcase
end
3'h6: begin //or
    Control = 15'b100000001100010;
end
3'h7: begin //and
    Control = 15'b100000001110010;
end

```

```

        end
        default: begin
            Control = 15'bz;
        end
    endcase
end
7'h63 : begin // B Type
    case (funct3)
        3'h0: begin //beq
            Control = 15'b0010011000000110;
        end
        3'h1: begin //bne
            Control = 15'b0010011000000110;
        end
        3'h4: begin //blt
            Control = 15'b0010011000000110;
        end
        3'h5: begin //bge
            Control = 15'b0010011000000110;
        end
        3'h6: begin //bltu
            Control = 15'b0010111000000110;
        end
        3'h7: begin //bgeu
            Control = 15'b0010111000000110;
        end
        default: begin
            Control = 15'bz;
        end
    endcase
end
7'h03 : begin // Load Instructions
    case (funct3)
        3'h0: begin //lb
            Control = 15'b1000001000000001;
        end
        3'h1: begin //lh
            Control = 15'b1000001000000001;

```

```

        end
        3'h2: begin //lw
            Control = 15'b100000100000001;
        end
        default: begin
            Control = 15'bz;
        end
    endcase
end
7'h23 : begin // Store Instructions
    case (funct3)
        3'h0: begin //sb
            Control = 15'b000100100001110;
        end
        3'h1: begin //sh
            Control = 15'b000100100001110;
        end
        3'h2: begin //sw
            Control = 15'b000100100001110;
        end
        default: begin
            Control = 15'bz;
        end
    endcase
end
7'h6F : begin // jal
    Control = 15'b110001100000100;
end
7'h67 : begin // jalr
    Control = 15'b100000100000100;
end
7'h17 : begin // auipc
    Control = 15'b101101100000010;
end
7'h37 : begin // lui
    Control = 15'b101101111110010;
end
default: begin

```

```

        Control = 14'bz;

    end

endcase

end

endmodule

```

这是一个将指令解码为各个控制信号的元件，根据指令需求给原件赋各个信号的值。

信号如下：

RegWEn: 寄存器写使能信号，为 1 则为可写。

ImmSel: 立即数生成解码选择信号，我们实现的 RISC-V 指令共有六种，除了 R 型指令不涉及立即数之外，剩下的指令涉及立即数，但对应的指令位数不同，所以分配了一个信号进行解码，信号分配如下：

[0b000 = I], [0b001 = S], [0b010 = B], [0b011 = U], [0b100 = J]

BrUn: 分支指令比较是否为无符号数的信号，若为 1 则表示寄存器中的数是无符号数。

ASel: ALU 的第一个操作数，若为 1 则为 PC，若为 0 则为寄存器文件读取的第一个源寄存器 rs1。

BSel: ALU 的第二个操作数，若为 1 则为立即数生成器输出的立即数，若为 0 则为寄存器文件读取的第二个源寄存器 rs2。

ALUSel: 控制 ALU 选择对应的运算符，见 ALU 部分。

MemWEn: 内存写信号，为 1 则为可写。

WBSel: 写回选择信号，为 0 则为内存读取结果，为 1 则为 ALU 选择结果，为 2 则为 PC+4。

MemREn: 内存读信号，为 1 则为可读。

根据每个指令的实际功能，依次填写了这个表格，并将其依次填入了代码中。

add rd, rs1, rs2	R	0b011100 11	0b000 0b00000000	0b0000000	0	1	000	0	0	0	0000	0	01	
mul rd, rs1, rs2			0b000 0b00000001	0b0000001	1	1	000	0	0	0	1000	0	01	0
sub rd, rs1, rs2			0b000 0b01000000	0b0000010	2	1	000	0	0	0	1100	0	01	0
sll rd, rs1, rs2			0b001 0b00000000	0b0000011	3	1	000	0	0	0	0001	0	01	0
mulh rd, rs1, rs2			0b001 0b00000001	0b0001000	4	1	000	0	0	0	1001	0	01	0
mulhu rd, rs1, rs2			0b011 0b00000001	0b000101	5	1	000	0	0	0	1011	0	01	0
sll rd, rs1, rs2			0b010 0b00000000	0b000110	6	1	000	0	0	0	0010	0	01	0
xor rd, rs1, rs2			0b100 0b00000000	0b000111	7	1	000	0	0	0	0100	0	01	0
srl rd, rs1, rs2			0b101 0b00000000	0b001000	8	1	000	0	0	0	0101	0	01	0
sra rd, rs1, rs2			0b101 0b01000000	0b001001	9	1	000	0	0	0	1101	0	01	0
or rd, rs1, rs2			0b110 0b00000000	0b001010	10	1	000	0	0	0	0110	0	01	0
and rd, rs1, rs2			0b111 0b00000000	0b001011	11	1	000	0	0	0	0111	0	01	0
lw rd, offset(rs1)	I	0b0000011	0b010	0b001110	14	1	000	0	0	1	0000	0	00	1
addi rd, rs1, imm			0b000	0b001111	15	1	000	0	0	1	0000	0	01	0
slli rd, rs1, imm			0b001 0b00000000	0b010000	16	1	000	0	0	1	0001	0	01	0
slti rd, rs1, imm			0b010	0b010001	17	1	000	0	0	1	0010	0	01	0
xori rd, rs1, imm			0b100	0b010010	18	1	000	0	0	1	0100	0	01	0
srti rd, rs1, imm			0b101 0b00000000	0b010011	19	1	000	0	0	1	0101	0	01	0
srai rd, rs1, imm			0b101 0b01000000	0b010100	20	1	000	0	0	1	1101	0	01	0
ori rd, rs1, imm			0b110	0b010101	21	1	000	0	0	1	0110	0	01	0
andi rd, rs1, imm			0b111	0b010110	22	1	000	0	0	1	0111	0	01	0
sw rs2, offset(rs1)			0b010 0b0100011	0b010	25	0	001	0	0	1	0000	1	11	0
beq rs1, rs2, offset			0b000	0b011010	26	0	010	0	1	1	0000	0	11	0
bne rs1, rs2, offset			0b001	0b011011	27	0	010	0	1	1	0000	0	11	0
blt rs1, rs2, offset	B	0b1100011	0b100	0b011100	28	0	010	0	1	1	0000	0	11	0
bge rs1, rs2, offset			0b101	0b011101	29	0	010	0	1	1	0000	0	11	0
bltu rs1, rs2, offset			0b110	0b011110	30	0	010	1	1	1	0000	0	11	0
oge rs1, rs2, offset			0b111	0b011111	31	0	010	1	1	1	0000	0	11	0
auipc rd, offset	U	0b0010111		0b1000000	32	1	011	0	1	1	0000	0	01	0
lui rd, offset				0b0110111	33	1	011	0	1	1	1111	0	01	0
jai rd, imm	J	0b1101111		0b100010	34	1	100	0	1	1	0000	0	10	0
jai rd, rs1, imm	I	0b1100111	0b000	0b100011	35	1	000	0	0	1	0000	0	10	0

(5) 寄存器文件:

```
`define DATA_WIDTH 32
module RegFile(
    CLK, WE3, RA1, RA2, WA3, WD3,
    RD1, RD2,
    X1, X2, X3, X4
);
    parameter ADDR_SIZE = 5;
    input CLK, WE3;
    input [ADDR_SIZE-1:0] RA1, RA2, WA3;
    input [`DATA_WIDTH-1:0] WD3;
    output [`DATA_WIDTH-1:0] RD1, RD2;
    output [`DATA_WIDTH-1:0] X1, X2, X3, X4;

    reg [`DATA_WIDTH-1:0] rf[2 ** ADDR_SIZE-1:0];

    always @(posedge CLK)
        if (WE3) rf[WA3] <= WD3;
    assign RD1 = (RA1 != 0) ? rf[RA1] : 0;
    assign RD2 = (RA2 != 0) ? rf[RA2] : 0;
    assign X1 = rf[1];
    assign X2 = rf[2];
    assign X3 = rf[3];
    assign X4 = rf[4];
endmodule
```

这里除了基本的寄存器文件的读写之外, 还设置了四个输出 X1, X2, X3, X4 用于读取寄存器 x1, x2, x3, x4 用于调试。

(6) ImmGen (立即数生成器)

```
module ImmGen(
    input [31:0] instruction,
    input [2:0] ImmSel,
    output reg [31:0] Immediate
);
    reg instr8;
    reg [3:0] instr12;
    reg [7:0] instr20;
    reg instr21;
    reg [3:0] instr25;
    reg [5:0] instr31;
    reg instr32;

    always@(*) begin
        instr8 = instruction[7];
        instr12 = instruction[11:8];
    end
```



```

instr20 = instruction[19:12];
instr21 = instruction[20];
instr25 = instruction[24:21];
instr31 = instruction[30:25];
instr32 = instruction[31];
case(ImmSel)
    3'b000: begin Immediate = {{21{instr32}}, instr31, instr25,
instr21}; end
    3'b001: begin Immediate = {{21{instr32}}, instr31, instr12,
instr8}; end
    3'b010: begin Immediate = {{20{instr32}}, instr8, instr31,
instr12, 1'b0}; end
    3'b011: begin Immediate = {instr32, instr31, instr25,
instr21, instr20, 12'b0}; end
    3'b100: begin Immediate = {{12{instr32}}, instr20, instr21,
instr31, instr25, 1'b0}; end
    default: begin Immediate = 32'bx; end
endcase
end

```

endmodule

用于对指令进行解码，输出对应指令中包含的立即数。RISC-V 不同指令的位数包含的信息如下：

	31	25 24	20 19	15 14	12 11	7 6	0
R	funct7	rs2	rs1	funct3	rd	opcode	
I	imm[11:0]		rs1	funct3	rd	opcode	
I*	funct7	imm[4:0]	rs1	funct3	rd	opcode	
S	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	
B	imm[12 10:5]	rs2	rs1	funct3	imm[4:1 11]	opcode	
U	imm[31:12]				rd	opcode	
J	imm[20 10:1 11 19:12]				rd	opcode	

Immediates are sign-extended to 32 bits, except in I* type instructions

(7) ALU (算数逻辑单元)

```

module ALU(
    OP, A, B, F
);
parameter SIZE = 32;
input [3:0] OP;
input signed [SIZE:1] A;
input signed [SIZE:1] B;
output [SIZE:1] F;
reg [SIZE:1] F;

```

```

reg [2*SIZE-1:0] mul_result;
reg [2*SIZE-1:0] unsigned_mul_result;
always@(*) begin
    mul_result = A * B;
    unsigned_mul_result = $unsigned(A) * $unsigned(B);
    case(OP)
        4'b0000: begin F = A + B; end
        4'b0001: begin F = A << B; end
        4'b0010: begin F = A < B ? 1 : 0; end
        4'b0100: begin F = A ^ B; end
        4'b0101: begin F = A >> B; end
        4'b0110: begin F = A | B; end
        4'b0111: begin F = A & B; end
        4'b1000: begin F = mul_result[31:0]; end
        4'b1001: begin F = mul_result[63:32]; end
        4'b1011: begin F = unsigned_mul_result[63:32]; end
        4'b1100: begin F = A - B; end
        4'b1101: begin F = A >>> B; end
        default: begin F = B; end
    endcase
end
endmodule

```

算数逻辑单元实现的运算如下：

ALUSel	Instruction
0	add: Result = A + B
1	sll: Result = A << B[4:0]
2	slt: Result = (A < B (signed)) ? 1 : 0
3	Unused
4	xor: Result = A ^ B
5	srl: Result = (unsigned) A >> B[4:0]
6	or: Result = A B
7	and: Result = A & B
8	mul: Result = (signed) (A * B)[31:0]
9	mulh: Result = (signed) (A * B)[63:32]
10	Unused
11	mulhu: Result = (A * B)[63:32]
12	sub: Result = A - B
13	sra: Result = (signed) A >> B[4:0]
14	Unused
15	bsel: Result = B

(7) 控制逻辑信号 (PCSel)

```

module ControlLogic_EX(
    input [31:0] instr,
    input BrEq, BrLt,

```

```

output PCSel,
output [2:0] ImmSel,
output BrUn, BSel, ASel,
output [3:0] ALUSel
);
wire Jump;
wire [5:0] Branch;
MainDsc_EX MainDsc_EX_1(instr, Jump, Branch);
assign PCSel = Jump | (Branch[5] & BrEq) | (Branch[4] & ~BrEq) |
(Branch[3] & BrLt) | (Branch[2] & ~BrLt) | (Branch[1] & BrLt) | (Branch[0]
& ~BrLt);
endmodule
module MainDsc_EX(
input [31:0] instr,
output Jump,
output [5:0] Branch
);
reg [6:0] op;
reg [2:0] funct3;
reg [6:0] Control;
assign {Jump, Branch} = Control;
always@(*) begin
op = instr[6:0];
funct3 = instr[14:12];
case (op)
7'h63 : begin // B Type
case (funct3)
3'h0: begin //beq
Control = 7'b01000000;
end
3'h1: begin //bne
Control = 7'b00100000;
end
3'h4: begin //blt
Control = 7'b00010000;
end
3'h5: begin //bge
Control = 7'b00001000;
end
3'h6: begin //bltu
Control = 7'b00000100;
end
3'h7: begin //bgeu
Control = 7'b00000001;

```

```

        end
        default: begin
            Control = 7'bz;
        end
    endcase
end
7'h6F : begin // jal
    Control = 7'b1000000;
end
7'h67 : begin // jalr
    Control = 7'b1000000;
end
default: begin
    Control = 7'b0;
end
endcase
end
endmodule

```

由于 PCSel 信号需要在 EX 阶段根据读出的源寄存器值，通过分支比较器进行解码，因此单独设计了一个元件。

这个信号主要负责判断是否属于跳转、分支指令，并判断下一个 PC 的值。

若信号为 0，则 $PC = PC + 4$ ；若信号为 1，则 $PC = \text{ALU 的结果}$ 。

(8) BranchComp 分支比较器

```

module BranchComp(
    input signed [31:0] RD1,
    input signed [31:0] RD2,
    input BrUn,
    output reg BrEq,
    output reg BrLt
);
always@(*) begin
    case (BrUn)
        1'b0: begin
            BrEq = RD1 == RD2;
            BrLt = RD1 < RD2;
        end
        1'b1: begin
            BrEq = $unsigned(RD1) == $unsigned(RD2);
            BrLt = $unsigned(RD1) < $unsigned(RD2);
        end
    endcase
end
endmodule

```

主要负责根据两个寄存器的值和 BrUn 信号输出 BrEq 信号（若相等输出 1，否则输出 0）和 BrLt 信号（若 RD1<RD2 输出 1，否则输出 0），用于判断分支指令是否采用分支。

（9）DMem（数据存储器）

```
module DMem(
    input [31:0] MemAddr,
    input [31:0] MemWriteData,
    input MemREn,
    input MemWEn,
    input CLK,
    output [31:0] MemReadData
);

    wire [127:0] BlockData;
    wire [31:0] cache_data;
    wire [31:0] WBAAddr;
    wire [31:0] mem_data;
    wire WB, miss;
    wire [127:0] WBDData;
    FourWaySACache Cache_1(CLK, MemAddr[11:0], MemWriteData, MemWEn,
MemREn, BlockData, cache_data, WB, miss, WBDData, WBAAddr);
    RAM_4Kx32 MainMem(MemAddr, MemWriteData, WBAAddr, WBDData, WB, MemREn
& miss, miss & MemWEn, CLK, mem_data, BlockData);
    assign MemReadData = miss ? mem_data : cache_data;
endmodule
```

这里实现了一个四路组相联缓存（1KB 大小，四路，16 个组，块大小 16 个字节）和一个简单的 4KBx32 位的主存。首先经过缓存，缓存命中则输出 miss = 0 不读取 RAM，否则输出 miss = 1 读取 RAM，输出对应读取值的同时输出整个块，用于覆盖缓存。

（10）四路组相联缓存

```
module FourWaySACache (
    input clk,
    input [11:0] address,
    input [31:0] write_data,
    input write_enable,
    input read_enable,
    input [127:0] block_data,
    output reg [31:0] read_data,
    output reg writeback,
    output reg miss,
    output reg [127:0] write_back_data,
    output reg [31:0] write_back_addr
);
```

```

parameter NUM_SETS = 16;
parameter NUM_WAYS = 4;

reg [3:0] tags [NUM_SETS-1:0][NUM_WAYS-1:0];
reg [127:0] data [NUM_SETS-1:0][NUM_WAYS-1:0]; // Data storage
reg valid [NUM_SETS-1:0][NUM_WAYS-1:0];
reg dirty [NUM_SETS-1:0][NUM_WAYS-1:0];
reg [1:0] lru [NUM_SETS-1:0][NUM_WAYS-1:0];

integer i;
integer j;
integer way_hit = -1;
integer replace_way = -1;
integer ost;
wire [3:0] tag = address[11:8];
wire [3:0] index = address[7:4];
wire [3:0] offset = address[3:0];

initial begin
    miss <= 0;
    writeback <= 0;
    for (i = 0; i < NUM_SETS; i = i + 1) begin
        for (j = 0; j < NUM_WAYS; j = j + 1) begin
            lru[i][j] = j;
            valid[i][j] = 0;
            tags[i][j] = 0;
            data[i][j] = 0;
            dirty[i][j] = 0;
        end
    end
end

always @(*) begin
    if (read_enable | write_enable) begin
        way_hit = -1;
        for (i = 0; i < NUM_WAYS; i = i + 1) begin
            if (valid[index][i] && tags[index][i] == tag) begin
                way_hit = i;
            end
        end

        if (way_hit == -1) begin
            miss <= 1;
        end
    end
end

```

```

        case ({read_enable, write_enable})
            2'b10: begin
                read_data <= data[index][way_hit][offset * 8 +:
32];

                miss <= 0;
            end
            2'b01: begin
                data[index][way_hit][offset * 8 +: 32] <=
write_data;

                dirty[index][way_hit] <= 1;
                miss <= 0;
            end
            default: begin
                read_data <= 32'bz;
            end
        endcase
    end
end
else begin
    read_data <= 32'bz;
end
end

always @(posedge clk) begin
    if (read_enable | write_enable) begin
        way_hit = -1;
        for (i = 0; i < NUM_WAYS; i = i + 1) begin
            if (valid[index][i] && tags[index][i] == tag) begin
                way_hit = i;
            end
        end

        if (way_hit == -1) begin
            replace_way = lru[index][0];
            for (i = 1; i < NUM_WAYS; i = i + 1) begin
                lru[index][i-1] <= lru[index][i];
            end
            lru[index][3] <= replace_way;
            if (dirty[index][replace_way]) begin
                write_back_data <= data[index][replace_way];
                write_back_addr <= {tags[index][replace_way], index,
4'b0000};

                writeback <= 1;
            end
        end
    end
end

```

```

        else begin
            writeback <= 0;
        end
        data[index][replace_way] <= block_data;
        valid[index][replace_way] <= 1;
        dirty[index][replace_way] <= 0;
        tags[index][replace_way] <= tag;
    end
    else begin
        writeback <= 0;
        if (lru[index][0] == way_hit) begin
            lru[index][0] <= lru[index][1];
            lru[index][1] <= lru[index][2];
            lru[index][2] <= lru[index][3];
        end
        if (lru[index][1] == way_hit) begin
            lru[index][1] <= lru[index][2];
            lru[index][2] <= lru[index][3];
        end
        if (lru[index][2] == way_hit) begin
            lru[index][2] <= lru[index][3];
        end
        lru[index][3] <= way_hit;
    end
end
else begin
    read_data <= 32'bz;
    writeback <= 0;
end
end
endmodule

```

这是一个采用 LRU 算法进行替换的写回缓存。

首先对地址进行分析，地址共有 12 位，作如下位数分析：

```

// 1KiB Cache Size
// 16B Block Size
// 12 bit Address
// Offset bit =  $\log(16) = 4$ 
// Sets Num =  $1024/16/4 = 16$ 
// Index bit =  $\log(16) = 4$ 
// Tag bit =  $12 - 4 - 4 = 4$ 

```

所以地址位数的 Tag: Index: Offset = 4 : 4 : 4

根据这个结果编写代码，首先对地址进行 Tag: Index: Offset 的解码，然后遍历对应 Index 的数据块寻找有效 (valid = 1) 且 tag 位相同的数据块，若找到，则缓存命中，设置 miss = 0，根据输入的读写使能信号，若读使能信号为 1，则

将输出 read_data 赋值为数据块中对应 Offset 的值；若写使能信号为 1，则在缓存中写入数据，并将 dirty（脏位）设置为 1，当缓存被替换时，若脏位为 1，则输出这个块的数据写回内存。另外，在时钟上升沿的时候，更新 LRU 的数组，将当前路数放到队尾，用于决定替换逻辑。

若缓存不命中，则将 miss 设置为 1 并输出，等待 RAM 读取并输出对应的数据块。在时钟周期上升沿的时候，决定替换的路数，这里根据 LRU 算法，总是选取最长时间不使用的路数，即 LRU 数组的队首（因为每次读取都将读取的路放到队尾）。替换后将其放到队尾。若发生替换，且替换的路的 dirty 位为 1，则将其写回主存。

(11) RAM

```
module RAM_4Kx32(
    input [31:0] Addr,
    input [31:0] Data_in,
    input [31:0] WriteBackAddr,
    input [127:0] WriteBackData,
    input WB,
    input CS,
    input WE,
    input CLK,
    output reg [31:0] Data_out,
    output reg [127:0] BlockData
);

parameter Addr_Width = 12;
parameter Data_Width = 32;
parameter SIZE = 2 ** Addr_Width;
integer i;
reg [Data_Width-1:0] RAM [SIZE-1:0];
initial begin
    for (i = 0; i < SIZE; i = i + 1) begin
        RAM[i] = 0;
    end
end
always @(*) begin
    casex({CS, WE})
        4'b10 : begin
            BlockData <= {RAM[Addr/4*4+3], RAM[Addr/4*4+2],
RAM[Addr/4*4+1], RAM[Addr/4*4]};
            Data_out <= RAM[Addr];
        end
        4'b01 : begin
            RAM[Addr] <= Data_in;
            if (WB) begin
                RAM[WriteBackAddr] <= WriteBackData[31:0];
            end
        end
    endcase
end
```

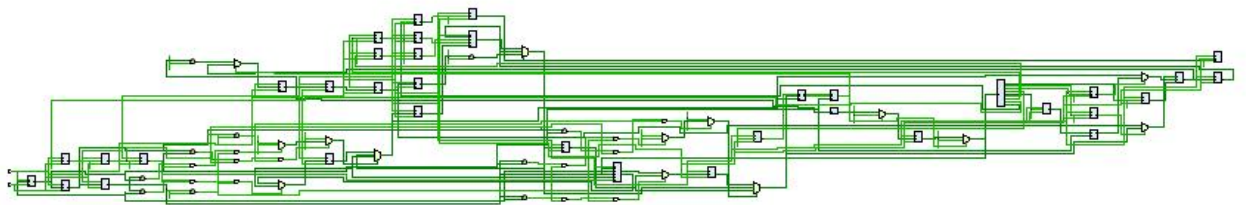
```

        RAM[WriteBackAddr+1] <= WriteBackData[63:32];
        RAM[WriteBackAddr+2] <= WriteBackData[95:64];
        RAM[WriteBackAddr+3] <= WriteBackData[127:96];
    end
end
default : Data_out <= 32'bz;
endcase
end
endmodule

```

这里实现了一个简单的 RAM，除了基本功能之外，也为缓存的脏写重新设计了结构，一个新的信号 WB 用于进行控制，若有效则将缓存被替换的块写回主存。

(12) RTL 分析



(13) 仿真结果

这里有了 3 个仿真代码：

①基本指令的测试

```

ORI x1, x0, 5
ORI x2, x0, 10
ADD x3, x1, x2
SUB x4, x2, x1
SW x3, 0(x1)
LW x4, 0(x1)
BEQ x3, x4, Label
LUI x4, 0x1234
NOP
Label:
NOP

```

对应的机器码为：

```

00506093
00a06113
002081b3
40110233
0030a023
0000a203
00418663

```

01234237

00000013

00000013

仿真结果：



注意到 x1 x2 x3 x4 四个寄存器的值变化都符合预期，且在分支预测失败之后，指令均被替换为气泡，功能符合预期。对于缓存，这里仅在 SW x3, 0(x1) 指令这里 miss 了一次，随后进行 LW x4, 0(x1) 的时候缓存命中，成功读取了缓存中对应的值。

②更多类型的指令与循环的测试

addi x1, x0, 1

sub x2, x1, x0

addi x4, x1, 3

slti x3, x1, 2

blt x2, x1, Label

Label:

addi x1, x1, 1

beq x1, x4, Label2

jal x2, Label

Label2:

lui x1, 100

sw x2, 0(x1)

```
lw x3, 0(x1)
对应的机器码:
00100093
40008133
00308213
0020A1B3
00114263
00108093
00408463
FF9FF16F
000640B7
0020A023
0000A183
仿真结果:
```



根据 RISC-V 指令，我们可以知道循环次数为 3 次，直到 $x1=x4$ ，代码中有三次分支预测失败，气泡插入，其中两次为分支指令，一次为跳转指令。由此，分支预测实现符合预期。寄存器的值的变化也符合程序预期。

缓存方面，与示例 1 类似，在 sw 指令时 miss，随后 lw 指令 hit，从缓存中成功读取数据。

③缓存测试

```

addi x1 x0 100
addi x2 x0 200
addi x3 x0 300
addi x4 x0 400
sw x1 0(x0)
sw x2 256(x0)
sw x3 512(x0)
sw x4 768(x0) // miss, 写入缓存

lw x4 0(x0)
lw x3 256(x0)
lw x2 512(x0)
lw x1 768(x0) // hit

lw x4 0(x0)
lw x3 256(x0)
lw x2 512(x0) // hit, 将替换目标设置为 768(x0)

addi x4 x4 100
sw x4 1024(x0) // miss, 替换 768(x0)

lw x4 0(x0)
lw x3 256(x0)
lw x2 512(x0)
lw x1 1024(x0) //到这里为止都是 hit
lw x4 768(x0) //miss, 替换 0(x0)

addi x4 x4 100
sw x4 16(x0)
lw x3 16(x0) //测试其他组的缓存

addi x4 x4 100
sw x4 0(x0) //miss, 替换 256(x0)

sw x4 1280(x0)
sw x4 1536(x0)
sw x4 1792(x0)
sw x4 768(x0) //到这里为止全是 miss, 替换。

```

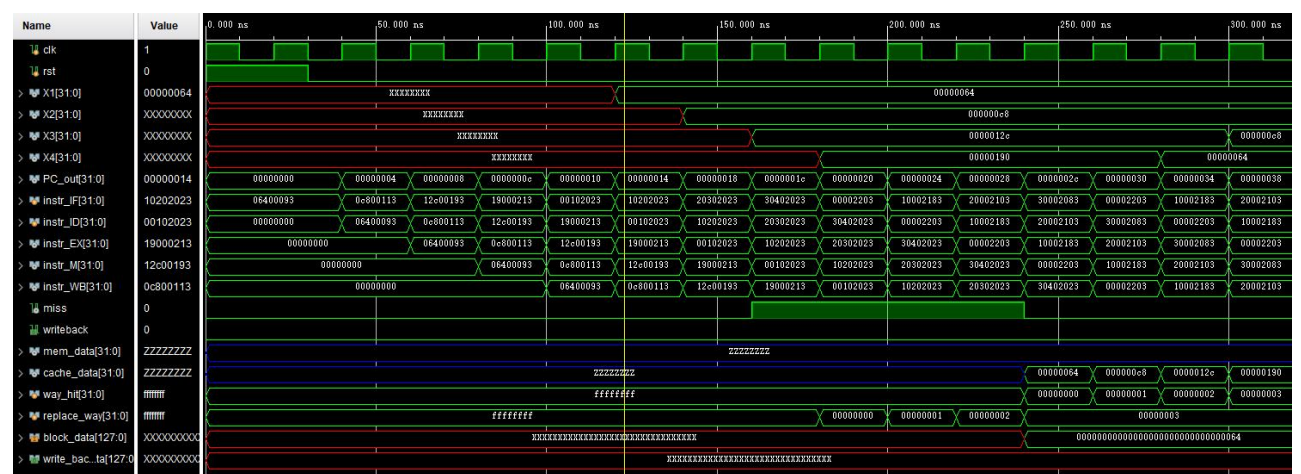
对应机器码:

```

06400093
0c800113
12c00193
19000213

```

仿真结果:





如图所示，miss 信号的输出符合预期，且读取的数据正确，缓存正常工作。

五、调试过程与心得

因为本学期前往加州大学伯克利分校进行交流，因此这次实验大体是基于 UC Berkeley CS61C 课程的 Project3 进行实现的，在此基础之上添加了五级流水线和四路组相联缓存（原实验仅要求实现两级的简单流水线）。

在实现 CS61C 课程 Project3 刚开始实验的时候，我先花了些时间理解 RISC 指令集的格式和功能需求。然后用 Verilog 实现了一些基本模块，比如寄存器堆、ALU、指令存储器等。刚搭建好这些模块的时候，我对整个流程还不是特别清晰，只能一步步查资料，根据课程上讲解的数据通路模型一步步实现。

做多周期 CPU 的时候，指令执行被拆分成了多个阶段，刚开始实现的时候，调试十分复杂，令人头痛。最后通过单元测试跑了一遍又一遍，直到每个阶段的结果都符合预期。

流水线实现起来难度高很多。最麻烦的是数据冒险和控制冒险。在未加转发单元和分支预测单元的情况下，流水线 CPU 的运行结果不正确。加入流水线并添加调试后才终于符合预期。

缓存部分是我觉得最困难的环节。刚开始写 4 路组相联缓存的时候，连基本逻辑都没理顺，经常在模拟过程中看到缓存读写出错。后来一步步拆解功能，分别实

现地址映射、替换策略和写策略，最后终于使得缓存符合预期。

虽然实验完成了，但这个 CPU 还不够完美，比如分支预测可以用更高级的预测，缓存的替换策略也能更智能，还有一个选做内容超标量未能实现。这些内容虽然没时间深入研究，但给我留下了很多思考，也激发了我对计算机体系结构更深入学习的兴趣。

总的来说，这次实验虽然过程挺曲折，但收获很大。我不仅对硬件设计有了更多理解，也感受到调试和耐心的重要性。如果以后有机会，我希望能做更多类似的项目，把这次学到的东西用得更好。